

**LAPORAN PRAKTIKUM TUGAS AKHIR PENGEMBANGAN
APLIKASI BERBASIS WEB
Modul 4: Mengoperasikan Database dan Generate Data Otomatis
dengan Laravel**



**Universitas
Telkom**

Disusun Oleh:

Nama: Dryhus Dzacky Damingtyas.S

NIM: 707012530010

Kelas: D4SIKC-49-04

Mata Kuliah: Pengembangan Aplikasi Berbasis Web

**UNIVERSITAS
TELKOM FAKULTAS
ILMU TERAPAN
JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA
CERDAS TAHUN AJARAN 2026**

Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

Menghubungkan project LaporanBanjir dengan database MySQL melalui pengaturan pada file .env.

Membuat tabel laporan memakai file migration beserta method up() dan down().

Membuat model Eloquent Laporan yang terhubung ke tabel laporan.

Mengubah controller, route, dan view supaya data dari form pelaporan bisa disimpan ke database.

Menampilkan kembali data laporan yang sudah tersimpan pada halaman daftar laporan.

Mengisi tabel laporan dengan data dummy secara otomatis memakai seeder dan library Faker.

Alat dan Bahan

Praktikum ini dikerjakan pada laptop dengan spesifikasi berikut.

Perangkat Keras

Laptop dengan prosesor 13th Gen Intel Core i7-13620H.

RAM 16 GB (terbaca 15,7 GB).

Sistem operasi Windows 11 Home Single Language (build 10.0.26200).

Perangkat Lunak

XAMPP 8.2.12, di dalamnya terdapat Apache 2.4.58, MariaDB 10.4.32, PHP 8.2.12, dan phpMyAdmin 5.2.1.

Composer 2.10.3 sebagai dependency manager PHP.

Laravel Framework 12.69.3.

Visual Studio Code 1.140.0 sebagai teks editor.

Google Chrome 154.0.8037.97 untuk menjalankan aplikasi di browser.

Git 2.53.0 untuk version control.

C. Langkah-Langkah Praktikum

Praktikum ini melanjutkan project LaporanBanjir yang dibuat pada Modul 3. Pada modul sebelumnya data laporan masih ditulis sebagai array di dalam controller, sehingga laporan hilang begitu halaman ditutup. Di modul ini data laporan disimpan ke database MySQL supaya bisa dibuka kembali kapan saja. Pekerjaan dilakukan berurutan, mulai dari menyiapkan database sampai mengisi tabel dengan data contoh.

C.1 Konsep Migration Laravel

Migration adalah fitur Laravel yang mencatat riwayat perubahan struktur database, kira-kira seperti version control untuk database. Setiap perubahan tabel ditulis dalam satu file migration, sehingga struktur database bisa dibangun ulang di komputer lain hanya dengan menjalankan perintah migrate. File migration memiliki dua method, yaitu up() untuk membuat atau mengubah tabel dan down() untuk membatalkan perubahan saat rollback.

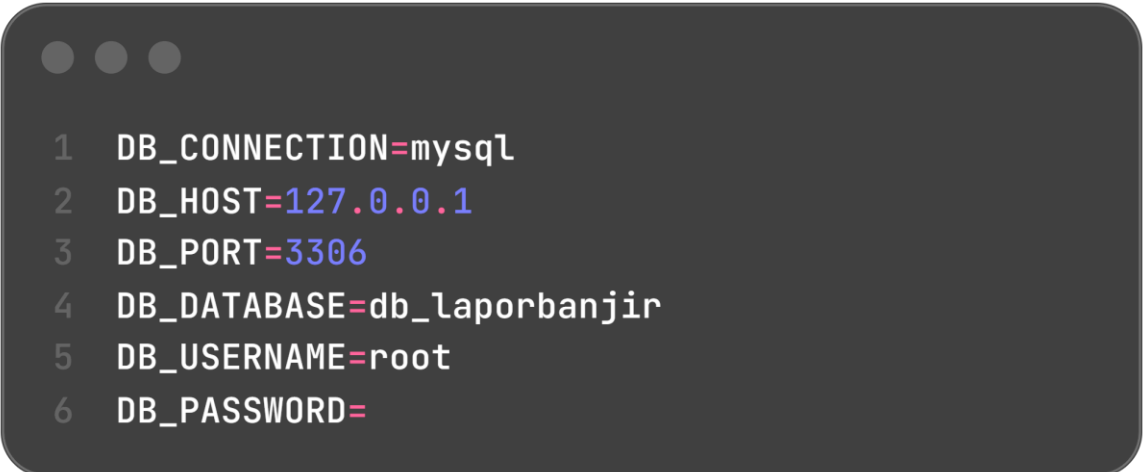
Ada beberapa alasan migration dipakai. Perubahan tabel bisa dibuat dan diubah dengan cepat, file migration bisa dikirim ke programmer lain supaya struktur database tetap konsisten, dan programmer tidak perlu lagi melakukan export atau import file SQL secara manual. Kalau terjadi kesalahan, perubahan terakhir bisa dikembalikan dengan perintah migrate:rollback.

C.2 Konfigurasi Database

Langkah pertama adalah menyalakan Apache dan MySQL pada XAMPP Control Panel sampai keduanya berwarna hijau. Apache berperan sebagai web server, sedangkan MySQL menyimpan data aplikasi.

Setelah itu saya membuka phpMyAdmin di <http://localhost/phpmyadmin>, memilih menu New, lalu membuat database baru bernama db_laporbanjir. Database ini masih kosong karena tabelnya akan dibuat melalui migration.

Koneksi database diatur pada file .env di folder utama project. Bagian yang saya ubah adalah konfigurasi DB seperti berikut.



```
1 DB_CONNECTION=mysql
2 DB_HOST=127.0.0.1
3 DB_PORT=3306
4 DB_DATABASE=db_laporbanjir
5 DB_USERNAME=root
6 DB_PASSWORD=
```

Nilai DB_CONNECTION diubah menjadi mysql karena Laravel versi terbaru secara bawaan memakai sqlite. DB_DATABASE diisi db_laporbanjir, DB_USERNAME diisi root, dan DB_PASSWORD dibiarkan kosong mengikuti pengaturan default XAMPP.

C.3 Membuat Migration

Tabel laporan dibuat lewat migration, bukan langsung di phpMyAdmin. Saya menjalankan perintah berikut di terminal.

```
php artisan make:migration create_laporan_table
```

Perintah itu membuat file baru di folder database/migrations dengan nama berawalan tanggal dan waktu, contohnya 2026_10_05_131323_create_laporan_table.php. Awalan waktu tersebut dipakai Laravel untuk menentukan urutan eksekusi migration.

Isi file migration kemudian saya ubah menjadi seperti di bawah ini.

```

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('laporan', function (Blueprint $table) {
15             $table->id();
16             $table->string('nama_pelapor');
17             $table->string('lokasi');
18             $table->integer('tinggi_genangan');
19             $table->date('tanggal_kejadian');
20             $table->timestamps();
21         });
22     }
23
24     /**
25      * Reverse the migrations.
26      */
27     public function down(): void
28     {
29         Schema::dropIfExists('laporan');
30     }
31 };
32

```

Method up() berisi Schema::create('laporan', ...) yang membuat tabel laporan beserta kolomnya. Kolom id() menjadi primary key, nama_pelapor bertipe string, lokasi bertipe string, tinggi_genangan bertipe integer untuk menyimpan tinggi air dalam satuan cm, tanggal_kejadian bertipe date, dan timestamps() menambahkan kolom created_at serta updated_at. Method down() berisi Schema::dropIfExists('laporan') untuk menghapus tabel saat rollback.

Setelah file disimpan, migration dijalankan dengan perintah php artisan migrate. Perintah ini mengeksekusi semua file migration yang belum pernah dijalankan. Selain tabel laporan, Laravel juga membuat tabel bawaan seperti users, cache, dan jobs, serta tabel migrations yang mencatat migration mana saja yang sudah dieksekusi.

Hasilnya saya periksa di phpMyAdmin dan tabel laporan sudah terbentuk dengan kolom id, nama_pelapor, lokasi, tinggi_genangan, tanggal_kejadian, created_at, dan updated_at.

C.4 Membuat Model

Model adalah class yang mewakili satu tabel di database. Lewat model, data bisa disimpan dan diambil tanpa menulis perintah SQL secara langsung karena Laravel memakai Eloquent ORM. Model dibuat dengan perintah berikut.

php artisan make:model Laporan

Perintah itu menghasilkan file `app/Models/Laporan.php`. Nama model ditulis dalam bentuk tunggal dan diawali huruf kapital. Isinya kemudian saya ganti dengan kode berikut.

```
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class Laporan extends Model
8  {
9      protected $table = 'laporan';
10
11      protected $fillable = [
12          'nama_pelapor',
13          'lokasi',
14          'tinggi_genangan',
15          'tanggal_kejadian',
16      ];
17  }
18
```

Properti `$table` diisi 'laporan' supaya model terhubung ke tabel yang benar. Nama tabel pada modul ini berbentuk tunggal, sedangkan secara bawaan Laravel menganggap nama tabel adalah bentuk jamak dari nama model, sehingga pengaturan ini perlu ditulis manual. Properti `$fillable` berisi daftar kolom yang boleh diisi sekaligus lewat mass assignment, yaitu `nama_pelapor`, `lokasi`, `tinggi_genangan`, dan `tanggal_kejadian`. Kalau ada kolom yang tidak dicantumkan di `$fillable`, Laravel akan menolak pengisiannya dan menampilkan `MassAssignmentException`.

C.5 Mengubah Controller

Controller menjadi penghubung antara view dan model. Controller menerima permintaan pengguna, meminta model menyimpan atau mengambil data, lalu mengirim hasilnya ke view. Pada Modul 3 controller `LaporanController` sudah ada, tetapi datanya masih berupa array. Di modul ini isi controller saya sesuaikan supaya memakai model. File yang diubah adalah `app/Http/Controllers/LaporanController.php`.

```

1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\Laporan;
6  use Illuminate\Http\Request;
7
8  class LaporanController extends Controller
9  {
10     public function index()
11     {
12         $laporan = Laporan::all();
13
14         return view('laporan.index', compact('laporan'));
15     }
16
17     public function create()
18     {
19         return view('laporan.form');
20     }
21
22     public function store(Request $request)
23     {
24         $laporan = Laporan::create([
25             'nama_pelapor' => $request->nama_pelapor,
26             'lokasi' => $request->lokasi,
27             'tinggi_genangan' => $request->tinggi_genangan,
28             'tanggal_kejadian' => $request->tanggal_kejadian,
29         ]);
30
31         return view('laporan.konfirmasi', compact('laporan'));
32     }
33 }
34

```

Di dalam controller ada tiga method. Method `index()` mengambil seluruh data dengan `Laporan::all()` lalu mengirimkannya ke view daftar laporan. Method `create()` menampilkan halaman form. Method `store()` menyimpan data dari form ke database lewat `Laporan::create()` dan meneruskan data yang baru tersimpan ke halaman konfirmasi. Model `Laporan` diimpor di bagian atas file supaya bisa dipakai, sedangkan data dari form dibaca melalui objek `Request`. Berbeda dengan Modul 3 yang datanya masih berupa array, di modul ini data benar-benar disimpan ke dan diambil dari database.

C.6 Routing

Route menghubungkan alamat URL dengan method pada controller. Route untuk `LaporBanjir` masih memakai alamat yang sama seperti Modul 3, hanya saja aksinya sekarang menuju

database. Saya menyesuaikan file routes/web.php.

```
1  <?php
2
3  use Illuminate\Support\Facades\Route;
4  use App\Http\Controllers\LaporanController;
5
6  Route::get('/', function () {
7      return view('welcome');
8  });
9
10 Route::get('/laporan', [LaporanController::class, 'index'])->name('laporan.index');
11 Route::get('/laporan/create', [LaporanController::class, 'create'])->
12 >name('laporan.create');
13 Route::post('/laporan', [LaporanController::class, 'store'])->name('laporan.store');
```

Route /laporan memakai Route::get untuk menampilkan daftar laporan, route /laporan/create memakai Route::get untuk menampilkan form, dan route /laporan memakai Route::post untuk menerima data yang dikirim form. Controller juga perlu diimpor di bagian atas file dengan use App\Http\Controllers\LaporanController; agar bisa dikenali. Ketiga route diberi nama memakai name() supaya bisa dipanggil dari view dengan route().

C.7 Membuat View Form

View adalah bagian yang dilihat pengguna. Form pelaporan pada Modul 3 hanya punya tiga input, sedangkan tabel baru menambahkan kolom tanggal_kejadian. Karena itu form saya sesuaikan agar setiap input cocok dengan nama kolom di tabel. File yang diubah adalah resources/views/laporan/form.blade.php.

```

1  @extends('layouts.app')
2
3  @section('title', 'Form Pelaporan')
4
5  @section('content')
6      <h2>Form Pelaporan Banjir</h2>
7
8      <form action="{{ route('laporan.store') }}" method="POST">
9          @csrf
10
11          <p>
12              <label>Nama Pelapor</label><br>
13              <input type="text" name="nama_pelapor">
14          </p>
15
16          <p>
17              <label>Lokasi Kejadian</label><br>
18              <input type="text" name="lokasi">
19          </p>
20
21          <p>
22              <label>Tinggi Genangan Air (cm)</label><br>
23              <input type="number" name="tinggi_genangan">
24          </p>
25
26          <p>
27              <label>Tanggal Kejadian</label><br>
28              <input type="date" name="tanggal_kejadian">
29          </p>
30
31          <button type="submit">Kirim Laporan</button>
32      </form>
33  @endsection
34

```

Form dikirim dengan method POST ke action route laporan.store. Directive `@csrf` ditulis di dalam form untuk melindungi dari serangan CSRF. Atribut name pada tiap input, yaitu `nama_pelapor`, `lokasi`, `tinggi_genangan`, dan `tanggal_kejadian`, dipakai controller untuk membaca data lewat `$request->nama_pelapor` dan seterusnya. Nama atribut ini sebaiknya sama dengan nama kolom pada tabel dan isi `$fillable`.

C.8 Membuat View Daftar Laporan

Halaman daftar laporan tidak lagi memakai array. Data diambil dari database lewat method `index()` dan ditampilkan dengan `@forelse`. File yang diubah adalah `resources/views/laporan/index.blade.php`.

```

1  @extends('layouts.app')
2
3  @section('title', 'Daftar Laporan')
4
5  @section('content')
6      <h2>Daftar Laporan Banjir</h2>
7
8      @forelse($laporan as $item)
9          @include('partials.laporan-card', ['laporan' => $item])
10     @empty
11         <x-alert type="error" message="Belum ada laporan banjir." />
12     @endforelse
13 @endsection
14

```

Setiap laporan dirender melalui partial `resources/views/partials/laporan-card.blade.php`. Partial ini juga menentukan status genangan memakai `@if`, `@elseif`, dan `@else`.

```

1  <div style="border: 1px solid #ccc; background-color: #ffffff; padding: 15px; margin-
    bottom: 15px;">
2      <h3 style="margin: 0 0 5px 0;">{{ $laporan->nama_pelapor }}</h3>
3      <p style="margin: 0 0 5px 0;">Lokasi: {{ $laporan->lokasi }}</p>
4      <p style="margin: 0 0 5px 0;">Tinggi Genangan: {{ $laporan->tinggi_genangan }} cm</p>
5      <p style="margin: 0 0 5px 0;">Tanggal Kejadian: {{ $laporan->tanggal_kejadian }}</p>
6
7      <p style="margin: 0;">
8          Status:
9          @if($laporan->tinggi_genangan < 30)
10             <strong style="color: green;">Waspada</strong>
11          @elseif($laporan->tinggi_genangan <= 70)
12             <strong style="color: orange;">Siaga</strong>
13          @else
14             <strong style="color: red;">Awat</strong>
15          @endif
16      </p>
17 </div>
18

```

Karena data dari database berbentuk objek, kolomnya dipanggil dengan tanda panah seperti `$laporan->nama_pelapor`, bukan `$laporan['nama_pelapor']` seperti pada array di Modul 3. Ketentuan status yang dipakai tetap sama: tinggi kurang dari 30 cm Waspada, 30 sampai 70 cm Siaga, dan lebih dari 70 cm Awat. Halaman ini memakai layout `layouts/app.blade.php` dan komponen `<x-alert>` yang sudah dibuat pada Modul 3.

C.9 Halaman Konfirmasi

Halaman konfirmasi tetap dipakai setelah data disimpan. Bedanya, data yang ditampilkan bukan lagi array dari request, melainkan objek Laporan hasil penyimpanan. Halaman ini juga memakai komponen `<x-alert>` dengan tipe success untuk menandakan laporan berhasil dikirim. File-nya ada di `resources/views/laporan/konfirmasi.blade.php`.

C.10 Menjalankan Aplikasi

Server Laravel dijalankan dengan perintah `php artisan serve`. Server berjalan di alamat `http://127.0.0.1:8000` dan terminal harus tetap terbuka selama aplikasi dipakai. Apache dan MySQL pada XAMPP juga harus tetap berjalan.

Setelah server jalan, saya membuka `http://127.0.0.1:8000/laporan/create` untuk mengisi form, lalu `http://127.0.0.1:8000/laporan` untuk melihat daftar. Hasil pengujian dan kendala yang muncul dibahas pada bagian D dan E.

C.11 Membuat Seeder

Setelah tabel dibuat, isinya masih kosong. Untuk mengisi data contoh dalam jumlah banyak, Laravel menyediakan database seeding. Seeder disimpan di folder `database/seeder` dan dijalankan dengan `php artisan db:seed`. Migration bisa dibayangkan sebagai membangun rak, sedangkan seeder adalah kegiatan mengisi rak tersebut dengan buku.

Saya membuat seeder dengan perintah berikut.

```
php artisan make:seeder LaporanSeeder
```

Perintah itu membuat file `database/seeder/LaporanSeeder.php`. Di folder yang sama juga ada `DatabaseSeeder.php`, yaitu seeder utama bawaan Laravel yang bisa memanggil seeder lain lewat `$this->call(LaporanSeeder::class);` di dalam method `run()`.

C.12 Membuat Data Dummy dengan Faker

Supaya tidak perlu menulis data satu per satu, saya memakai Faker. Faker adalah library PHP yang sudah terpasang bersama Laravel dan bisa menghasilkan data palsu seperti nama orang dan angka. Faker mendukung locale `id_ID` sehingga nama yang dihasilkan terdengar lokal. Isi `LaporanSeeder.php` saya ganti dengan kode berikut.

```

1  <?php
2
3  namespace Database\Seeders;
4
5  use App\Models\Laporan;
6  use Illuminate\Database\Seeder;
7
8  class LaporanSeeder extends Seeder
9  {
10     public function run(): void
11     {
12         $faker = fake('id_ID');
13
14         $kecamatan = [
15             'Dayeuhkolot',
16             'Baleendah',
17             'Bojongsoang',
18             'Rancaekek',
19             'Majalaya',
20         ];
21
22         for ($i = 0; $i < 15; $i++) {
23             Laporan::create([
24                 'nama_pelapor' => $faker->name(),
25                 'lokasi' => 'Kecamatan '.$faker->randomElement($kecamatan),
26                 'tinggi_genangan' => $faker->numberBetween(10, 150),
27                 'tanggal_kejadian' => $faker->dateTimeBetween('-1 month', 'now')-
28                 >format('Y-m-d'),
29             ]);
30         }
31     }
32 }

```

Objek Faker dibuat dengan `fake('id_ID')` supaya formatnya Indonesia. Perulangan `for ($i = 0; $i < 15; $i++)` menghasilkan 15 data. Setiap baris disimpan ke tabel laporan memakai `Laporan::create([...])`. Kolom `nama_pelapor` diisi `$faker->name()`, lokasi dipilih acak dari daftar kecamatan di Kabupaten Bandung, yaitu Dayeuhkolot, Baleendah, Bojongsoang, Rancaekek, dan Majalaya, memakai `randomElement`. Tinggi genangan diisi angka acak 10 sampai 150 dengan `numberBetween`, dan tanggal kejadian diambil acak dalam satu bulan terakhir dengan `dateTimeBetween('-1 month', 'now')`.

Sebelum seeder dijalankan, migration sudah lebih dulu dijalankan supaya tabel laporan tersedia. Setelah itu seeder dijalankan dengan `php artisan db:seed --class=LaporanSeeder`. Kedua langkah itu sebenarnya bisa digabung menjadi `php artisan migrate:fresh --seed`. Setiap kali seeder dijalankan, Laravel menambahkan 15 data baru. Karena datanya acak, nama pelapor dan tanggal yang muncul akan berbeda di tiap komputer. Pada project ini seeder dijalankan sekali sehingga tabel laporan berisi 15 baris data dengan status genangan yang bervariasi.

D. Hasil dan Dokumentasi Program

Berikut tangkapan layar dari aplikasi yang sudah dijalankan dan dari phpMyAdmin.

The screenshot shows the phpMyAdmin interface with the 'Table structure' tab selected for the 'laporan' table. The table has 7 columns: id (bigint(20), UNSIGNED, AUTO_INCREMENT), nama_pelapor (varchar(255), utf8mb4_unicode_ci), lokasi (varchar(255), utf8mb4_unicode_ci), tinggi_genangan (int(11)), tanggal_kejadian (date), created_at (timestamp, NULL), and updated_at (timestamp, NULL). The 'id' column is the primary key.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	nama_pelapor	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
3	lokasi	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
4	tinggi_genangan	int(11)			No	None			Change Drop More
5	tanggal_kejadian	date			No	None			Change Drop More
6	created_at	timestamp			Yes	NULL			Change Drop More
7	updated_at	timestamp			Yes	NULL			Change Drop More

Below the table structure, there are sections for 'Indexes' (showing a PRIMARY index on 'id'), 'Partitions' (No partitioning defined), and 'Information' (Space usage and Row statistics).

Gambar 1. Struktur tabel laporan di phpMyAdmin setelah migration dijalankan.

The screenshot shows the phpMyAdmin interface with the 'Structure' tab selected, displaying a list of tables in the 'db_laporbanjir' database. There are 10 tables listed: cache, cache_locks, failed_jobs, jobs, job_batches, laporan, migrations, password_reset_tokens, sessions, and users. The 'laporan' table is highlighted.

Table	Action	Rows	Type	Collation	Size	Overhead
cache	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K1B	-
cache_locks	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K1B	-
failed_jobs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K1B	-
jobs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 K1B	-
job_batches	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K1B	-
laporan	Browse Structure Search Insert Empty Drop	16	InnoDB	utf8mb4_unicode_ci	16.0 K1B	-
migrations	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	16.0 K1B	-
password_reset_tokens	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 K1B	-
sessions	Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_unicode_ci	48.0 K1B	-
users	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	32.0 K1B	-
10 tables	Sum	27	InnoDB	utf8mb4_unicode_ci	272.0 K1B	0 B

Below the table list, there is a 'Create new table' section with fields for 'Table name' and 'Number of columns' (set to 4).

Gambar 2. Daftar tabel pada database db_laporbanjir di phpMyAdmin.

LaporBanjir

Sistem Pelaporan Banjir BPBD Kabupaten Bandung

Form Pelaporan Daftar Laporan

Form Pelaporan Banjir

Nama Pelapor

Lokasi Kejadian

Tinggi Genangan Air (cm)

Tanggal Kejadian

dd/mm/yyyy

Kirim Laporan

© 2026 LaporBanjir - BPBD Kabupaten Bandung

Gambar 3. Halaman form pelaporan pada alamat /laporan/create.

LaporBanjir

Sistem Pelaporan Banjir BPBD Kabupaten Bandung

Form Pelaporan Daftar Laporan

Form Pelaporan Banjir

Nama Pelapor

Dedi Supriadi

Lokasi Kejadian

Kecamatan Baleendah

Tinggi Genangan Air (cm)

65

Tanggal Kejadian

04/10/2026

Kirim Laporan

© 2026 LaporBanjir - BPBD Kabupaten Bandung

Gambar 4. Form pelaporan yang sudah diisi sebelum dikirim.

Konfirmasi Laporan

Success — Laporan berhasil dikirim!

Data Laporan

Nama Pelapor: Dedi Supriadi

Lokasi Kejadian: Kecamatan Baleendah

Tinggi Genangan Air: 65 cm

Tanggal Kejadian: 2026-10-04

Gambar 5. Halaman konfirmasi setelah laporan berhasil disimpan.

LaporBanjir

Sistem Pelaporan Banjir BPBD Kabupaten Bandung

[Form Pelaporan](#) [Daftar Laporan](#)

Daftar Laporan Banjir

Ismail Marpaung

Lokasi: Kecamatan Bojongsoang
Tinggi Genangan: 132 cm
Tanggal Kejadian: 2026-09-23
Status: **Awas**

Nabila Lili Astuti

Lokasi: Kecamatan Bojongsoang
Tinggi Genangan: 14 cm
Tanggal Kejadian: 2026-09-16
Status: **Waspada**

Nyoman Jayadi Irawan M.Ak

Lokasi: Kecamatan Dayeuhkolot
Tinggi Genangan: 121 cm
Tanggal Kejadian: 2026-09-20
Status: **Awas**

Zamira Safitri

Lokasi: Kecamatan Baleendah
Tinggi Genangan: 131 cm
Tanggal Kejadian: 2026-09-29
Status: **Awas**

Titin Rahayu M.Farm

Lokasi: Kecamatan Baleendah
Tinggi Genangan: 125 cm
Tanggal Kejadian: 2026-09-14
Status: **Awas**

Respati Widodo

Lokasi: Kecamatan Bojongsoang
Tinggi Genangan: 144 cm
Tanggal Kejadian: 2026-09-25
Status: **Awas**

Darmanto Narpati

Lokasi: Kecamatan Majalaya
Tinggi Genangan: 18 cm
Tanggal Kejadian: 2026-09-26
Status: **Waspada**

Laras Padmasari

Lokasi: Kecamatan Dayeuhkolot
Tinggi Genangan: 50 cm
Tanggal Kejadian: 2026-09-18
Status: **Siaga**

Gambar 6. Halaman daftar laporan yang datanya diambil dari database.

Showing rows 0 - 15 (16 total, Query took 0.0006 seconds)

SELECT * FROM `laporan`

Number of rows: 25 | Filter rows: Search this table | Sort by key: None

	id	nama_pelapor	lokasi	tinggi_genangan	tanggal_kejadian	created_at	updated_at
☐	1	Ismail Marpaung	Kecamatan Bojongsong	132	2026-09-23	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	2	Nabila Lili Astuti	Kecamatan Bojongsong	14	2026-09-16	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	3	Nyoman Jayadi Irawan M Ak	Kecamatan Dayeuhkolot	121	2026-09-20	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	4	Zamira Safitri	Kecamatan Baleendah	131	2026-09-29	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	5	Titin Rahayu M Farm	Kecamatan Baleendah	125	2026-09-14	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	6	Respati Widodo	Kecamatan Bojongsong	144	2026-09-25	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	7	Darmanto Narpati	Kecamatan Majalaya	18	2026-09-26	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	8	Laras Padmasari	Kecamatan Dayeuhkolot	50	2026-09-18	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	9	Amalia Yulianti	Kecamatan Rancaekek	96	2026-09-23	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	10	Legawa Prasasta S Ked	Kecamatan Bojongsong	106	2026-09-13	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	11	Cinthia Widiastuti M Kom	Kecamatan Rancaekek	41	2026-09-06	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	12	Jaya Cahyadi Putra S Sos	Kecamatan Bojongsong	11	2026-09-26	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	13	Zamira Utami	Kecamatan Bojongsong	142	2026-09-14	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	14	Abyasa Permedi	Kecamatan Majalaya	45	2026-09-11	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	15	Jail Marbun M Kom	Kecamatan Rancaekek	51	2026-09-06	2026-10-05 13:14:01	2026-10-05 13:14:01
☐	17	Dryhus Dzacky Damingtyas S	Bojong	10	2008-02-21	2026-10-05 13:22:30	2026-10-05 13:22:30

Query results operations: Print, Copy to clipboard, Export, Display chart, Create view

Gambar 7. Data hasil seeder pada tabel laporan di phpMyAdmin.

E. Analisis dan Pembahasan

E.1 Peran Migration dan Seeder

Migration dan seeder punya tugas yang berbeda meskipun keduanya berhubungan dengan database. Migration mengurus struktur tabel, misalnya menentukan nama tabel dan tipe tiap kolom. Seeder mengurus isi tabel, yaitu memasukkan data awal atau data contoh. Karena itu urutannya selalu migration dulu, baru seeder. Pada praktikum ini migration membuat tabel laporan, sedangkan LaporanSeeder mengisi tabel tersebut dengan 15 data dummy.

File migration juga menyimpan riwayat perubahan struktur. Kalau suatu saat kolom diganti, perubahan itu ditulis di file migration baru dan bisa dijalankan ulang di komputer lain dengan php artisan migrate. Seeder tidak menyimpan riwayat seperti itu, karena fungsinya hanya mengisi data dan bisa dijalankan berkali-kali tanpa mengubah struktur.

E.2 Perbedaan Aplikasi Sebelum dan Sesudah Memakai Database

Pada Modul 3, data laporan masih ditulis sebagai array di dalam controller. Data seperti itu hanya hidup selama halaman dibuka dan hilang ketika halaman ditutup atau server dimatikan. Di modul ini data disimpan ke tabel MySQL melalui model Eloquent, sehingga data tetap ada meskipun server dijalankan ulang.

Perubahan lain terlihat pada cara kerja controller dan view. Method store() sekarang memanggil Laporan::create() untuk menulis baris baru ke tabel, sedangkan method index() memakai Laporan::all() untuk membaca seluruh data. Di sisi view, data dari database berbentuk objek sehingga kolomnya dipanggil dengan \$laporan->nama_pelapor, bukan \$laporan['nama_pelapor'] seperti pada array. Laravel juga menerjemahkan perintah Eloquent menjadi SQL secara otomatis, jadi tidak ada perintah INSERT atau SELECT yang ditulis manual.

E.3 Kendala dan Penyelesaian

1. Koneksi database masih mengarah ke sqlite. Laravel 12 secara bawaan memakai sqlite, sehingga sebelum aplikasi bisa menyimpan ke MySQL saya harus mengubah DB_CONNECTION menjadi mysql dan mengisi DB_DATABASE dengan db_laporbanjir pada file .env. Setelah itu koneksi berjalan dan migration bisa dijalankan.
2. MySQL belum berjalan saat praktikum dimulai. Apache dan MySQL pada XAMPP harus dinyalakan lebih dulu supaya Laravel bisa terhubung ke database. Hal yang sama berlaku untuk Apache karena phpMyAdmin diakses lewat web server tersebut.
3. Penamaan tabel tidak sama dengan dugaan bawaan Laravel. Nama model Laporan kalau dibiarkan akan dianggap Laravel sebagai tabel laporans, sedangkan modul meminta tabel bernama laporan. Penyelesaiannya adalah menulis properti \$table = 'laporan' pada model agar tidak salah tabel.
4. Batas status genangan. Nilai 30 cm dan 70 cm harus masuk kategori Siaga, jadi pengecekan status memakai @if(\$laporan->tinggi_genangan < 30), lalu @elseif(\$laporan->tinggi_genangan <= 70), dan @else untuk Awas. Dengan urutan itu batas bawah 30 cm dan batas atas 70 cm masuk ke kategori yang tepat.

F. Kesimpulan

1. Koneksi database Laravel berhasil diatur melalui file .env dengan DB_CONNECTION mysql dan DB_DATABASE db_laporbanjir, sehingga aplikasi bisa membaca dan menulis ke database MySQL.
2. Migration berhasil membuat tabel laporan beserta kolomnya, dan model Eloquent Laporan berhasil menghubungkan tabel itu ke aplikasi lewat properti \$table dan \$fillable.
3. Alur form, route, controller, model, dan database sudah tersambung. Laporan yang dikirim lewat form benar-benar tersimpan, dan halaman daftar laporan menampilkan data dari database dengan status genangan Waspada, Siaga, dan Awas.
4. Seeder dengan Faker locale id_ID berhasil mengisi tabel laporan dengan 15 data dummy yang bervariasi, sehingga tampilan halaman daftar bisa diuji tanpa perlu memasukkan data satu per satu.
5. Praktikum ini menunjukkan pentingnya menjaga kesesuaian antara struktur tabel dan kode yang memakainya, terutama pada nama kolom, isi \$fillable, dan nama input pada form.

G. Link Repository Github

Kode program lengkap praktikum ini dapat diakses pada repositori GitHub berikut:
<https://github.com/Dryhus21/PABW.git>